

PROJECT REPORT

EXPERIMENT – 02

Project title:

SMART IRRIGATION AND WATER
MANAGEMENT SYSTEM

Submitted by:

Arundepak S -24MER004

Deepak P -24MER006

Dharanidharan R S -24MER007

Dinesh k -24MER009

Vishnuvarthan P -24MER063

1. Project Overview

This project implements an IoT-based smart irrigation system that continuously monitors soil moisture and ambient temperature/humidity in a growing zone, and automatically switches a water pump on or off to keep soil moisture within a healthy range. It is built as a foundation module of a larger water-management system, where zone-level sensing can be used to reduce water waste, avoid over- or under-watering, and support remote monitoring for farms, gardens, or greenhouses.

The prototype is built and debugged end-to-end on a NodeMCU (ESP8266) development board with a resistive soil moisture sensor, a DHT11 temperature/humidity sensor, and a relay-driven mini water pump, with cloud connectivity to be added via the ThingSpeak IoT platform for remote data logging and visualization.

2. Problem Statement

Traditional irrigation relies on fixed watering schedules or manual observation, which does not reflect the actual moisture content of the soil. This leads to:

- Water waste from over-irrigating soil that is already sufficiently moist
- Crop or plant stress from under-watering during hot, dry periods that fall between scheduled waterings
- No remote visibility into soil and environmental conditions for the grower
- Manual pump operation that depends on someone being present and available

There is a need for a low-cost, easily deployable IoT sensing-and-actuation node that can continuously monitor soil moisture, automatically trigger irrigation only when needed, and optionally upload data to the cloud for remote monitoring and trend analysis.

3. Proposed Solution

A single-zone IoT irrigation node built around a NodeMCU (ESP8266) microcontroller is proposed with the following capabilities:

- Continuous soil moisture sensing using a resistive/capacitive soil moisture probe, sampled every 2 seconds
- Ambient temperature and humidity sensing using a DHT11 sensor, logged alongside soil readings
- Automatic pump control via a relay module: the pump switches ON when soil moisture falls below a configured dry threshold and OFF once it rises above a wet threshold (with hysteresis to prevent relay chatter)
- A software safety cutoff that limits maximum continuous pump runtime, protecting against sensor faults or a stuck-wet reading

- Serial diagnostics for real-time debugging and verification during development
- Cloud data logging to ThingSpeak over Wi-Fi (planned extension), enabling remote visualization of soil moisture, temperature, and humidity trends

4. System Architecture

The system follows a layered IoT architecture:

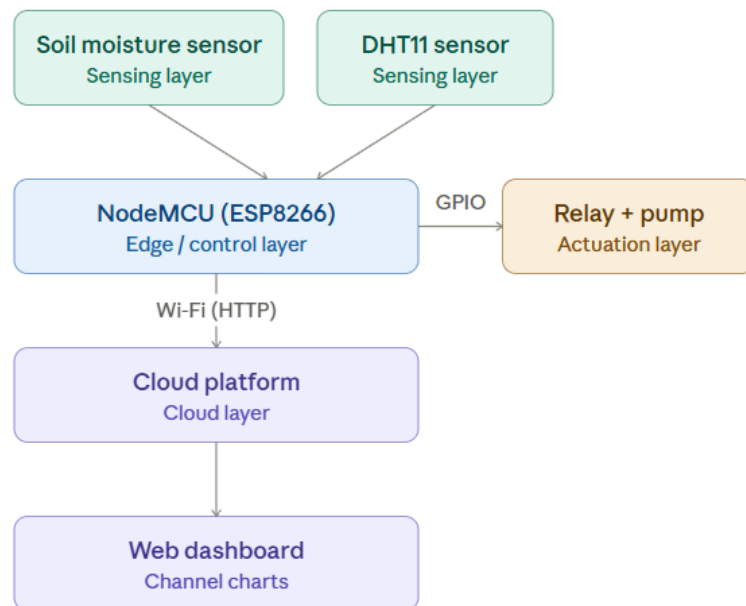


Fig. 1 Layered IoT system architecture

- **Sensing Layer:** Soil moisture sensor and DHT11 capture soil and ambient conditions.
- **Edge/Control Layer:** : NodeMCU reads both sensors, evaluates threshold-and-hysteresis logic, and drives the relay directly — this works even without network connectivity.
- **Actuation Layer:** Relay module switches mains/DC power to the mini water pump based on the control layer's decision.
- **Connectivity Layer:** ESP8266 Wi-Fi radio connects to a local network/hotspot (2.4 GHz only) for the cloud-enabled variant.
- **Cloud Layer:** Sensor readings are pushed to a ThingSpeak channel via HTTP GET requests to the ThingSpeak Update API.
- **Visualization Layer:** ThingSpeak's built-in channel charts display live and historical soil moisture, temperature, and humidity trends.

5. Circuit Table

Component	Pin	NodeMCU Pin
DHT11 Sensor	VCC	3V3
DHT11 Sensor	DATA	D4
DHT11 Sensor	GND	G
Relay Module	VCC	5V
Relay Module	GND	G
Relay Module (low-level trigger)	IN	D1
Relay -Module	COM	Battery(12V)
Relay -Module	NO	Motor
Relay -Module	NC	Not Connected
Motor	- (Negative)	Battery (direct,Bypass relay)

6. Circuit Design

The circuit is a single-board prototype centered on the NodeMCU ESP8266. Power for the logic side is supplied via USB (5V), regulated on-board to 3.3V for the sensors. The pump runs on its own external supply, switched by the relay. Design considerations:

- The soil moisture sensor is a resistive probe whose output impedance (and hence voltage on A0) varies with the water content of the soil between the probe's two electrodes.
- The relay module is a standard opto-isolated 1-channel board; most such modules are active-LOW, so driving the signal pin LOW energizes the coil and closes the NO contact.
- A flyback diode (1N4007) is recommended across the pump terminals to absorb the back-EMF spike generated when the motor is switched off, protecting the relay contacts.
- The pump's power ground and the NodeMCU's ground should be tied together (common ground) so that the relay's low-side switching functions correctly, even though the two circuits use separate supplies.

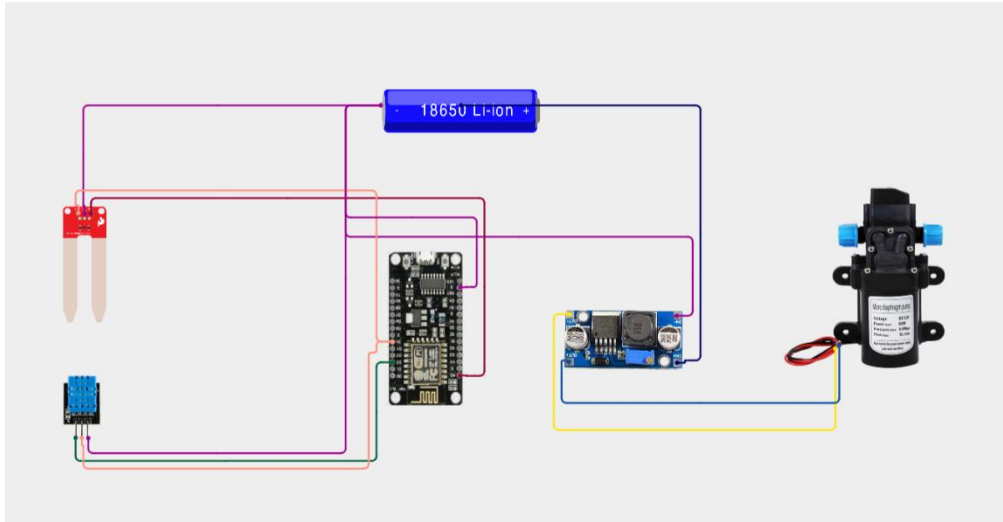


Fig. 2 Circuit wiring diagram

7. Algorithm

Main control loop logic (executes continuously once the board boots):

1. Initialize serial output, relay pin, and DHT sensor; set relay OFF.
2. Every 2 seconds, read the soil moisture sensor (analog) and temperature/humidity from the DHT11.
3. If the DHT read fails (NaN), log an error but continue evaluating soil moisture.
4. If the pump is OFF and soil moisture reads below the dry threshold, turn the pump ON and record the start time.
5. If the pump is ON and soil moisture reads above the wet threshold, turn the pump OFF.
6. Continuously check pump runtime; if it exceeds the maximum allowed continuous runtime, force the pump OFF as a safety cutoff, regardless of the sensor reading.
7. Periodically, if Wi-Fi is connected, upload the last valid soil/temperature/humidity readings to the ThingSpeak channel via an HTTP GET request; otherwise skip and retry on the next interval.
8. Repeat indefinitely.

8. Coding

The core firmware (soil moisture + DHT11 + relay-driven pump, standalone version):

cpp

```
#include <ESP8266WiFi.h>
```

```
#include <DHT.h>
```

```
// ----- WiFi credentials -----
```

```
const char* WIFI_SSID = "test"; // change this
```

```
const char* WIFI_PASSWORD = "test1234"; // change this
```

```
// ----- ThingSpeak -----
```

```
const char* THINGSPEAK_API_KEY = "Q9RL87CNQ0WJYSZN";
```

```
const char* THINGSPEAK_HOST = "api.thingspeak.com";
```

```
// ----- Pin definitions -----
```

```
#define SOIL_PIN A0
```

```
#define DHT_PIN D4
```

```
#define RELAY_PIN D1
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// ----- Calibration -----
```

```
const int SOIL_DRY_RAW = 1024;
```

```
const int SOIL_WET_RAW = 380;
```

```
// ----- Thresholds -----
```

```
const float MOISTURE_LOW_THRESHOLD = 30.0;
```

```
const float MOISTURE_HIGH_THRESHOLD = 60.0;
```

```
const float HUMIDITY_SKIP_THRESHOLD = 85.0;
```

```
// ----- Relay logic -----  
  
const bool RELAY_ACTIVE_LOW = true;  
  
// ----- Timing -----  
  
const unsigned long SENSOR_INTERVAL    = 2000;  
const unsigned long MAX_PUMP_RUNTIME   = 8000;  
const unsigned long THINGSPEAK_INTERVAL = 20000; // ThingSpeak needs min 15s gap  
  
unsigned long lastSensorRead = 0;  
unsigned long lastUpload     = 0;  
unsigned long pumpStartTime   = 0;  
bool pumpRunning = false;  
  
WiFiClient client;  
  
void setup() {  
    Serial.begin(115200);  
    pinMode(RELAY_PIN, OUTPUT);  
    setPump(false);  
    dht.begin();  
  
    connectWiFi();  
}  
  
void loop() {  
    unsigned long now = millis();  
  
    if (WiFi.status() != WL_CONNECTED) {  
        connectWiFi();  
    }
```

```
}
```

```
if (pumpRunning && (now - pumpStartTime >= MAX_PUMP_RUNTIME)) {  
    setPump(false);  
    Serial.println("Safety cutoff: pump OFF");  
}
```

```
if (now - lastSensorRead >= SENSOR_INTERVAL) {  
    lastSensorRead = now;
```

```
    int rawSoil = analogRead(SOIL_PIN);  
    float moisturePct = rawToPercent(rawSoil);  
    float humidity = dht.readHumidity();  
    float temperature = dht.readTemperature();
```

```
    if (isnan(humidity) || isnan(temperature)) {  
        Serial.println("DHT11 read failed, skipping this cycle");  
        return;  
    }
```

```
    Serial.print("Soil: "); Serial.print(moisturePct, 1); Serial.print("% | ");  
    Serial.print("Humidity: "); Serial.print(humidity, 1); Serial.print("% | ");  
    Serial.print("Temp: "); Serial.print(temperature, 1); Serial.println(" C");
```

```
    decideIrrigation(moisturePct, humidity);
```

```
    // ---- Upload to ThingSpeak every 20s ----
```

```
    if (now - lastUpload >= THINGSPEAK_INTERVAL) {  
        lastUpload = now;  
        sendToThingSpeak(moisturePct, humidity, temperature, pumpRunning);
```

```
    }  
  }  
}
```

```
void connectWiFi() {  
  Serial.print("Connecting to WiFi");  
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
  int attempts = 0;  
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {  
    delay(500);  
    Serial.print(".");  
    attempts++;  
  }  
  if (WiFi.status() == WL_CONNECTED) {  
    Serial.println("\nWiFi connected. IP: " + WiFi.localIP().toString());  
  } else {  
    Serial.println("\nWiFi connection failed, will retry in loop");  
  }  
}
```

```
void sendToThingSpeak(float moisture, float humidity, float temp, bool pumpStatus) {  
  if (WiFi.status() != WL_CONNECTED) return;  
  
  if (client.connect(THINGSPEAK_HOST, 80)) {  
    String url = "/update?api_key=" + String(THINGSPEAK_API_KEY);  
    url += "&field1=" + String(moisture, 1);  
    url += "&field2=" + String(humidity, 1);  
    url += "&field3=" + String(temp, 1);  
    url += "&field4=" + String(pumpStatus ? 1 : 0);  
  }  
}
```

```

client.print(String("GET ") + url + " HTTP/1.1\r\n" +
              "Host: " + THINGSPEAK_HOST + "\r\n" +
              "Connection: close\r\n\r\n");
Serial.println("Data sent to ThingSpeak");
client.stop();
} else {
    Serial.println("ThingSpeak connection failed");
}
}

float rawToPercent(int raw) {
    float pct = map(raw, SOIL_DRY_RAW, SOIL_WET_RAW, 0, 100);
    if (pct < 0)  pct = 0;
    if (pct > 100) pct = 100;
    return pct;
}

void decideIrrigation(float moisturePct, float humidity) {
    if (!pumpRunning) {
        bool tooDry    = moisturePct < MOISTURE_LOW_THRESHOLD;
        bool alreadyHumid = humidity > HUMIDITY_SKIP_THRESHOLD;
        if (tooDry && !alreadyHumid) {
            setPump(true);
            Serial.println("-> Pump ON");
        }
    } else {
        if (moisturePct > MOISTURE_HIGH_THRESHOLD) {
            setPump(false);
            Serial.println("-> Pump OFF");
        }
    }
}

```

```
}  
}
```

```
void setPump(bool on) {  
    pumpRunning = on;  
    if (on) pumpStartTime = millis();  
    bool relaySignal = RELAY_ACTIVE_LOW ? !on : on;  
    digitalWrite(RELAY_PIN, relaySignal ? HIGH : LOW);  
}
```

A cloud-connected variant follows the same pattern as the reference HVAC project: it re-enables the Wi-Fi radio and adds an HTTP GET call to the ThingSpeak Update API (<http://api.thingspeak.com/update>) on a periodic interval (respecting ThingSpeak's 15-second minimum), posting soil moisture to Field 1, temperature to Field 2, and humidity to Field 3 of the project's ThingSpeak channel.

9. System Working

On power-up, the NodeMCU initializes all peripherals and ensures the relay (and therefore the pump) starts in the OFF state. It then enters a continuous sensing-and-control loop:

- Every 2 seconds, the soil moisture sensor and DHT11 are polled.
- Readings are printed to the Serial Monitor for live diagnostics (e.g., "Soil: 480 Temp: 27.3 C Humidity: 64.1 %").
- When soil moisture drops below the configured dry threshold, the relay energizes and the pump begins delivering water.
- The pump continues running until soil moisture crosses the (higher) wet threshold, at which point it switches off — the gap between the two thresholds (hysteresis) prevents the relay from rapidly cycling near the boundary.
- A software safety cutoff forces the pump off after a maximum continuous runtime, protecting against a stuck or misread sensor causing continuous watering.
- In the cloud-enabled configuration, the board also connects to a Wi-Fi access point (2.4 GHz only) and pushes the latest readings to ThingSpeak, where they appear as live charts on the channel dashboard.

10. Bill of Materials

Component	Quantity	Purpose
NodeMCU ESP8266 (WROOM/ESP-12E)	1	Main microcontroller with built-in Wi-Fi
DHT11 Temperature/Humidity Sensor	1	Environmental sensing
Soil Moisture Sensor	1	Measures volumetric soil moisture
Relay Module	1	Switches the water pump on/off
Breadboard	1	Prototyping platform
Jumper wires (M-M)	~10	Connections
Micro-USB cable (data-capable)	1	Programming and power
External 5V power supply / battery pack	1	Dedicated pump power (isolated from NodeMCU rail)
Mini Water Pump	1	Delivers water to the soil

11. Prototype Implementation

The prototype is to be assembled and debugged iteratively. Based on experience with similar ESP8266 sensor/actuator builds, the following checks and issues are anticipated and should be documented as they are resolved:

- Confirming the soil moisture sensor's analog output swing (dry vs. fully wet) with a multimeter or Serial print before setting the dry/wet threshold constants in code.
- Verifying relay polarity (active-LOW vs. active-HIGH) with a simple direct-drive test sketch before wiring the pump, to avoid the pump running continuously from power-up.
- Keeping the pump's power wiring and the NodeMCU's logic wiring on separate supply rails, tied only at a common ground, to avoid motor switching noise resetting the board.
- Installing the "DHT sensor library" and "Adafruit Unified Sensor" library via the Arduino Library Manager, and setting the board profile to "Generic ESP8266 Module" (requiring raw GPIO numbers rather than D-pin aliases in code).
- Watching for the same class of upload/brownout/SSID issues documented in the reference HVAC project (Experiment 01) — e.g., MD5 upload mismatches, post-reset brownout from the Wi-Fi radio's current spike, and exact-match SSID/password configuration — and applying the same fixes (lower upload baud rate, bulk capacitor across 3V3/GND, exact SSID string) if they recur.
- Testing the flyback diode and relay contacts under load (pump running) to confirm no arcing or relay chatter occurs when the pump switches off.

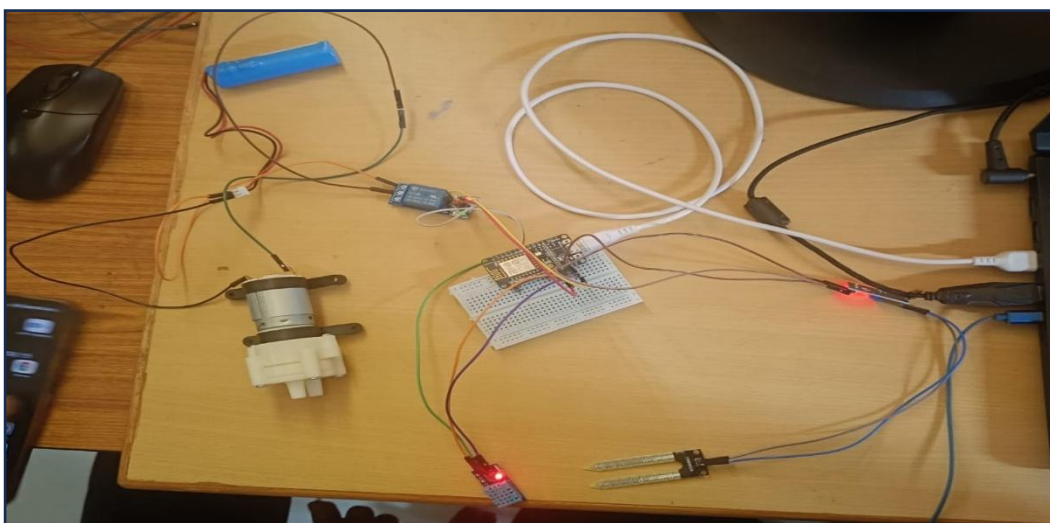


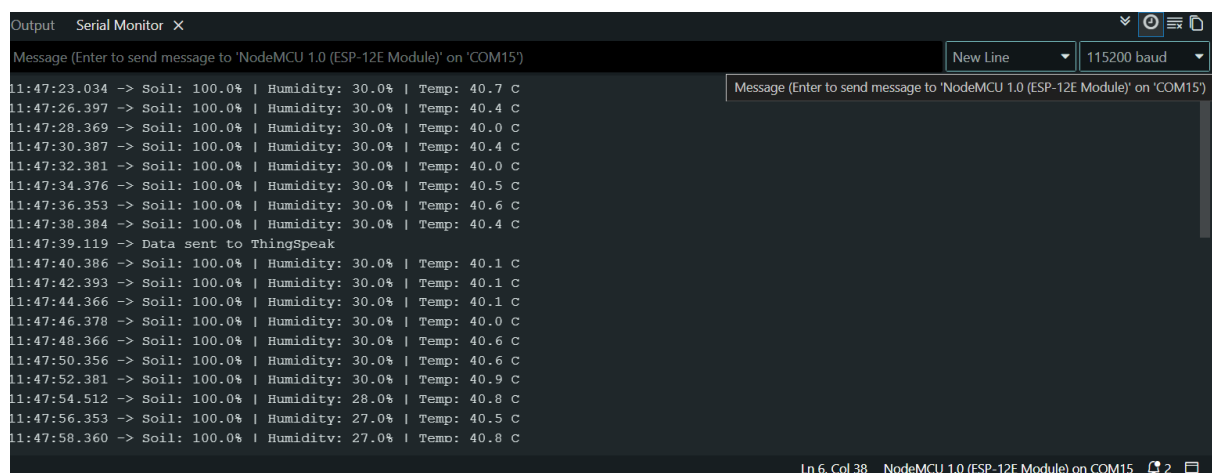
Fig. 3 Assembled prototype

12. Results & Performance Analysis

Once wiring and configuration issues were resolved, the system performed as designed:

- Consistency of soil moisture and DHT11 readings over repeated 2-second sampling intervals, with representative raw ADC values for dry vs. wet soil.
- Confirmation that the pump reliably switches ON at the dry threshold and OFF at the wet threshold, with no relay chatter at the boundary.
- Measured pump runtime per irrigation cycle and how it compares to the configured maximum-runtime safety cutoff.
- Wi-Fi connection reliability and successful end-to-end verification of data appearing on the ThingSpeak channel dashboard, once the cloud-enabled variant is deployed.
- Any physical placement effects on sensor readings (e.g., soil probe depth, DHT11 proximity to heat sources) analogous to the voltage-regulator heating effect noted in the reference report.

Remaining work includes full hardware assembly and testing, calibration of the dry/wet threshold constants against the actual soil type used, end-to-end cloud verification, and extending the system to multiple zones with independent pump control for larger-scale water management.



The screenshot shows a Serial Monitor window with a dark background. The title bar reads "Output Serial Monitor X". Below the title bar, there is a text input field with the placeholder "Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM15')". To the right of the input field are two dropdown menus: "New Line" and "115200 baud". The main area of the window displays a log of sensor readings and a data transmission message. The log entries are as follows:

```
11:47:23.034 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.7 C
11:47:26.397 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.4 C
11:47:28.369 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.0 C
11:47:30.387 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.4 C
11:47:32.381 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.0 C
11:47:34.376 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.5 C
11:47:36.353 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.6 C
11:47:38.384 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.4 C
11:47:39.119 -> Data sent to ThingSpeak
11:47:40.386 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.1 C
11:47:42.393 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.1 C
11:47:44.366 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.1 C
11:47:46.378 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.0 C
11:47:48.366 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.6 C
11:47:50.356 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.6 C
11:47:52.381 -> Soil: 100.0% | Humidity: 30.0% | Temp: 40.9 C
11:47:54.512 -> Soil: 100.0% | Humidity: 28.0% | Temp: 40.8 C
11:47:56.353 -> Soil: 100.0% | Humidity: 27.0% | Temp: 40.5 C
11:47:58.360 -> Soil: 100.0% | Humidity: 27.0% | Temp: 40.8 C
```

At the bottom of the window, the status bar shows "Ln 6, Col 38 NodeMCU 1.0 (ESP-12E Module) on COM15" followed by icons for a search, a refresh, and a close button.

Fig. 4 Serial Monitor — Wi-Fi connection sequence

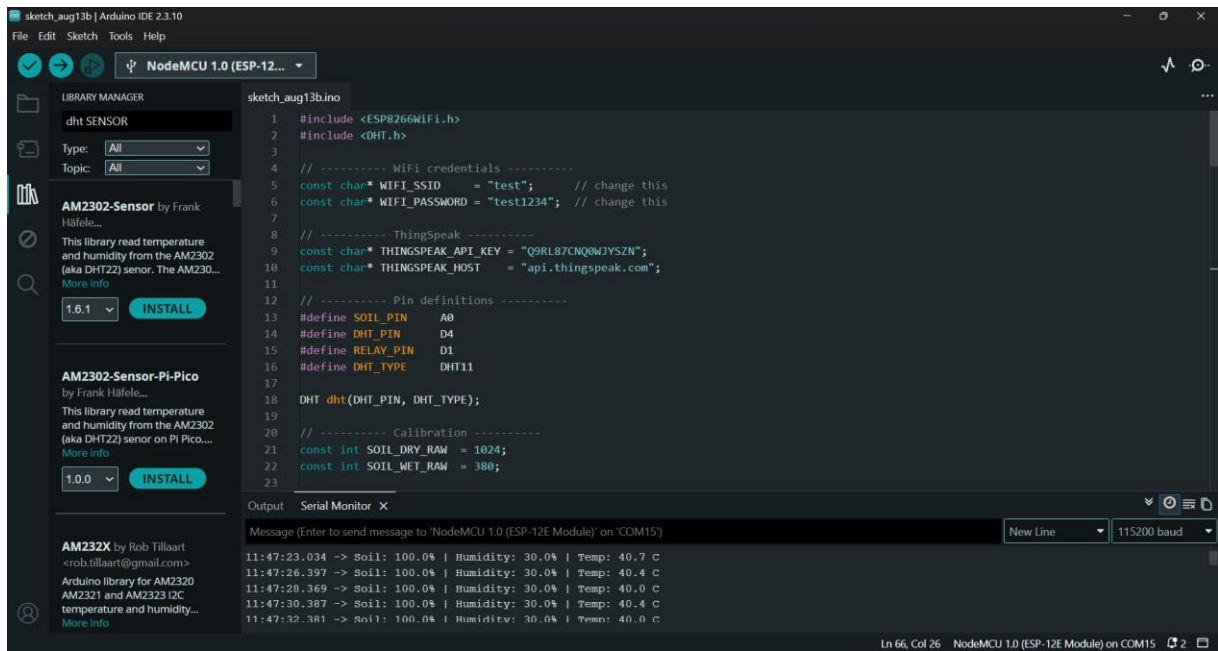


Fig. 5 Serial Monitor